

Architectural Justification

Alumni Mentorship & Mock Interview Platform

Architecture Selection: We chose Layered Architecture for the Alumni Mentorship & Mock Interview Platform System.

Reason 1 — Clear Separation for Distinct Academic Workflows

The platform serves two distinct actor groups (Student Mentees and Alumni Mentors) across multiple concerns — profile matching, session scheduling, and notifications. A Layered Architecture cleanly separates the Presentation Layer (touch/web UI used by both actors), the Business Layer (Matching Engine, Scheduling Service, Notification Service), and the Data Layer (profile, session, and feedback storage). This separation mirrors the university's existing academic-operations systems, making the platform easier for campus IT to understand, extend, and maintain over time as new features (e.g., group mentoring, resume review) are added.

Reason 2 — Predictable, Centralized Data Management

Mentor-mentee matching and interview scheduling both depend on a single, consistent view of student and alumni profiles, availability, and session history. A layered design routes all data access through a dedicated Data Layer, avoiding the data-consistency challenges that a more distributed style (such as Microservices) would introduce for a campus system of this scale. Since the platform is used within a single academic institution rather than at internet scale, the simpler, centrally coordinated data flow of a layered approach is a better fit than the added operational complexity of independently deployed services.

Security Advantage

The Layered Architecture enforces that the Presentation Layer never accesses the Database Component directly — all requests must pass through the Business Layer (Matching Engine, Scheduling Service, Notification Service). This gives the platform a single, well-defined checkpoint for authentication and authorization, ensuring that only verified Student Mentees and Alumni Mentors can view or modify profile, scheduling, and feedback data, and reducing the attack surface compared to designs that expose data services directly to clients.

Performance Benefit

Because related responsibilities are grouped within layers, the Business Layer components can cache frequently accessed data — such as mentor availability and matching results — before querying the Database Component again. This reduces redundant database calls during peak usage periods (e.g., when many students book mock interview slots ahead of a career fair), keeping response times low without requiring the added network overhead of a distributed, service-to-service architecture.